LINUX HOME LAB

SECURITY NETWORK

Complete Technical Documentation

PP

2nd Year

Tehničko veleučilište u Zagrebu (TVZ)

Table of Contents

Table of Contents

Table of Contents 2

1. Project Overview 4

1.1 Project Objectives 4

1.2 Security Architecture Philosophy 4

2. Network Architecture 6

2.1 Topology Overview 6

2.2 Network Segments 6

2.3 Machine Specifications 6

2.4 VirtualBox Internal Network Configuration 6

2.5 VirtualBox Network Adapter Configuration 7

pfSense Firewall — Two Adapters 7

Ubuntu Server — Single LAN Adapter 7

Kali Linux — Single WAN Adapter 7

3. Phase 1 — Network Topology Setup 9

3.1 pfSense Firewall Installation 9

3.2 Ubuntu Server Installation and Configuration 10

3.3 Kali Linux Installation and Configuration 11

3.4 Connectivity Verification 12

4. Phase 2 — Firewall Configuration 14

4.1 Understanding Default Deny 14

4.2 Disabling Private Network Blocking 14

4.3 Firewall Rules — WAN Interface 15

4.4 NAT Port Forwarding Rules 16

4.5 Verification — Default Deny in Action 19

HTTP (Port 80) — ALLOWED 19

HTTPS (Port 443) — BLOCKED 20

5. Phase 3 — Intrusion Detection System (Suricata) 21
5.1 Installation 21
5.2 Configuration 21
5.3 Custom Detection Rules 23
5.4 Rule Analysis 23
5.5 Attack Simulation and Detection Results 24
6. Phase 4 — WireGuard VPN 26
6.1 How WireGuard Encryption Works 26
6.2 Key Generation 26
6.3 Server Configuration (Ubuntu) 27
6.4 Client Configuration (Kali) 27
6.5 Tunnel Activation and Verification 27
6.6 VPN Status Verification 29
7. Problems Encountered and Lessons Learned 30
7.1 Default Route Conflicts During Package Installation 30
7.2 YAML Formatting Errors in Netplan 30
7.3 Forgotten Password and Linux Recovery 31
8. Tools and Technologies 32
9. Key Takeaways and Skills Demonstrated 33
9.1 Network Engineering 33
9.2 Security Engineering 33
9.3 Linux System Administration 33
9.4 Professional Development 34
10. Glossary of Terms 35

1. Project Overview

This document provides a complete technical walkthrough of a network security lab built using virtualized Linux systems and enterprise-grade open-source tools. The project demonstrates the practical implementation of key security concepts that form the backbone of modern enterprise network defense: network segmentation, firewall policy enforcement, real-time intrusion detection, and encrypted remote access via VPN.

The lab was built entirely in Oracle VirtualBox using three purpose-configured virtual machines: a pfSense firewall acting as the sole gateway between network zones, an Ubuntu Server running Suricata IDS for threat monitoring and WireGuard for VPN connectivity, and a Kali Linux machine simulating an external attacker. All systems communicate exclusively over isolated internal networks that replicate a real-world WAN/LAN separation found in corporate environments.

This project was developed as a personal learning initiative to move beyond network simulation (Cisco Packet Tracer) into hands-on implementation with real, running systems. Every component was installed, configured, tested, and troubleshot manually — providing deep practical experience that no simulation can fully replicate.

1.1 Project Objectives

- Design and implement a segmented network architecture with isolated WAN and LAN zones
- Configure a pfSense firewall enforcing a strict default deny policy with granular allow rules
- Deploy Suricata IDS with custom-written detection rules for real-time network threat monitoring
- Establish an encrypted WireGuard VPN tunnel enabling secure remote access to internal resources
- Simulate realistic attacks from Kali Linux and verify detection and logging by the IDS
- Document the entire build process for portfolio use, demonstrating practical security engineering skills

1.2 Security Architecture Philosophy

The lab implements a defense-in-depth strategy with three distinct security layers working together:

- **Layer 1 — Network Segmentation:** Physical separation of trusted and untrusted zones forces all traffic through a single controlled chokepoint (the firewall).
- **Layer 2 — Firewall Policy:** pfSense enforces a default deny policy — all traffic is blocked unless an explicit rule allows it. NAT port forwarding controls which internal services are exposed.

- **Layer 3 — Intrusion Detection:** Suricata monitors traffic that the firewall allows, looking for suspicious patterns like port scans, brute-force attempts, and unauthorized access within permitted traffic flows.

Bonus Layer — Encryption: WireGuard VPN ensures that even traffic traversing the untrusted WAN network is encrypted end-to-end, protecting data confidentiality.

2. Network Architecture

2.1 Topology Overview

2.2 Network Segments

Segment	Subnet	Gateway	Purpose
WAN	10.0.0.0/24	10.0.0.1	Untrusted external network (simulates the internet)
LAN	192.168.1.0/24	192.168.1.1	Protected internal network with monitored services
VPN Tunnel	10.10.10.0/24	—	Encrypted overlay network (WireGuard)
Router Link	10.0.0.0/24 ↔ 192.168.1.0/24	pfSense	Firewall-controlled inter-zone routing

2.3 Machine Specifications

Machine	Operating System	IP Address(es)	Role	Resources
pfSense-Firewall	FreeBSD (pfSense)	WAN: 10.0.0.1 LAN: 192.168.1.1	Firewall, Router, NAT	1 GB RAM, 8 GB disk
Ubuntu-IDS-Server	Ubuntu Server 24.04 LTS	LAN: 192.168.1.10 VPN: 10.10.10.1	IDS, VPN Server	1 GB RAM, 15 GB disk
Kali-Attacker	Kali Linux (Rolling)	WAN: 10.0.0.2 VPN: 10.10.10.2	Penetration Testing	2 GB RAM, 20 GB disk

2.4 VirtualBox Internal Network Configuration

Two VirtualBox internal networks were created to enforce strict zone separation:

- **intnet-wan** — Connects pfSense Adapter 1 (WAN) and Kali Adapter 1. This simulates the public internet / untrusted zone.
- **intnet-lan** — Connects pfSense Adapter 2 (LAN) and Ubuntu Adapter 1. This is the protected internal zone.

By using VirtualBox internal networks (not bridged or NAT), the lab is completely isolated from the host machine and real network. This is both a security best practice for lab environments and ensures that all traffic flows exclusively through the designed architecture.

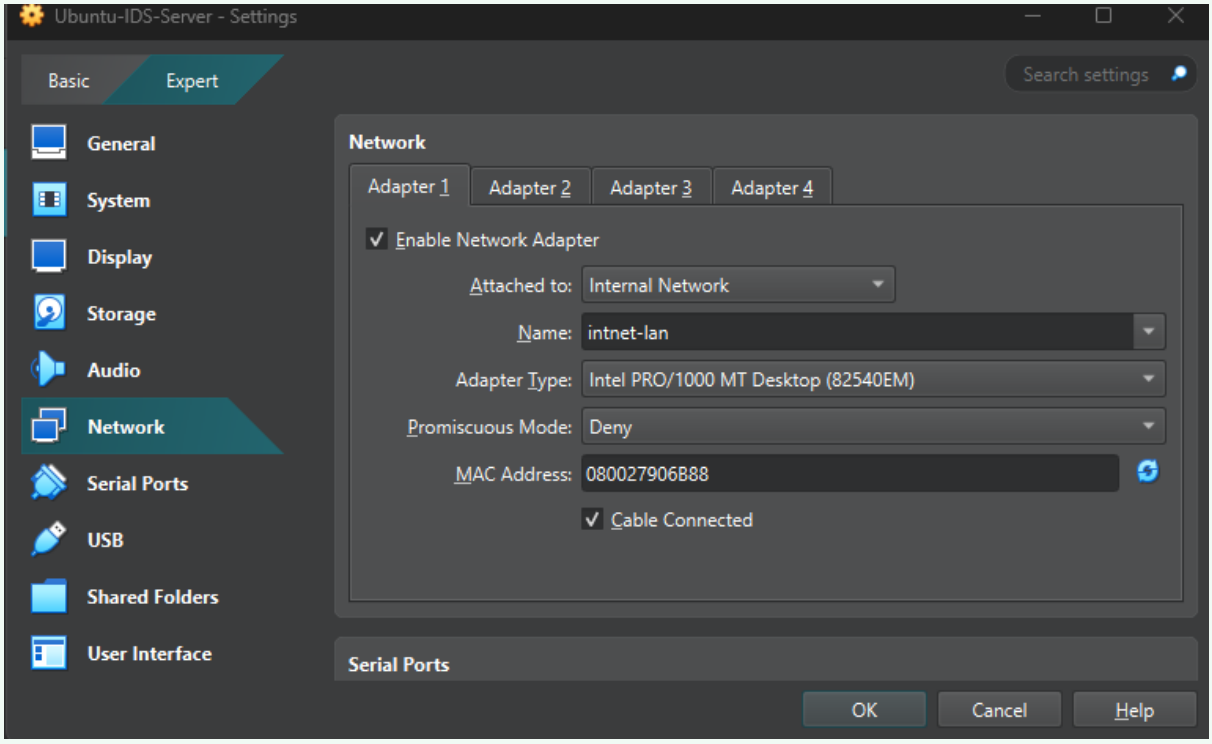
2.5 VirtualBox Network Adapter Configuration

Each virtual machine's network adapters were configured in VirtualBox Settings → Network before the operating system was installed. The adapter assignments are critical — a single typo in the internal network name would prevent the VMs from communicating.

pfSense Firewall — Two Adapters

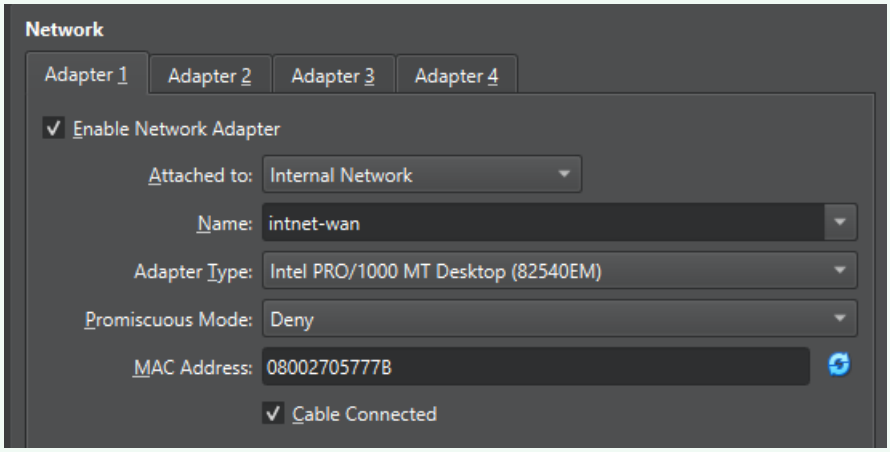
Ubuntu Server — Single LAN Adapter

Ubuntu Server has only one network adapter connected to "intnet-lan". As an internal server, it should only be reachable from the protected LAN zone — any external access must pass through the pfSense firewall first.



Kali Linux — Single WAN Adapter

Kali has one network adapter connected to "intnet-wan". As the simulated attacker, it sits on the untrusted external network and can only reach internal resources if the firewall permits it.


Network
Adapter 1
Adapter 2
Adapter 3
Adapter 4
Enable Network Adapter
Attached to:
Internal Network
Name:
intnet-wan
Adapter Type:
Intel PRO/1000 MT Desktop (82540EM)
Promiscuous Mode:
Deny
MAC Address:
08002705777B
Cable Connected

3. Phase 1 — Network Topology Setup

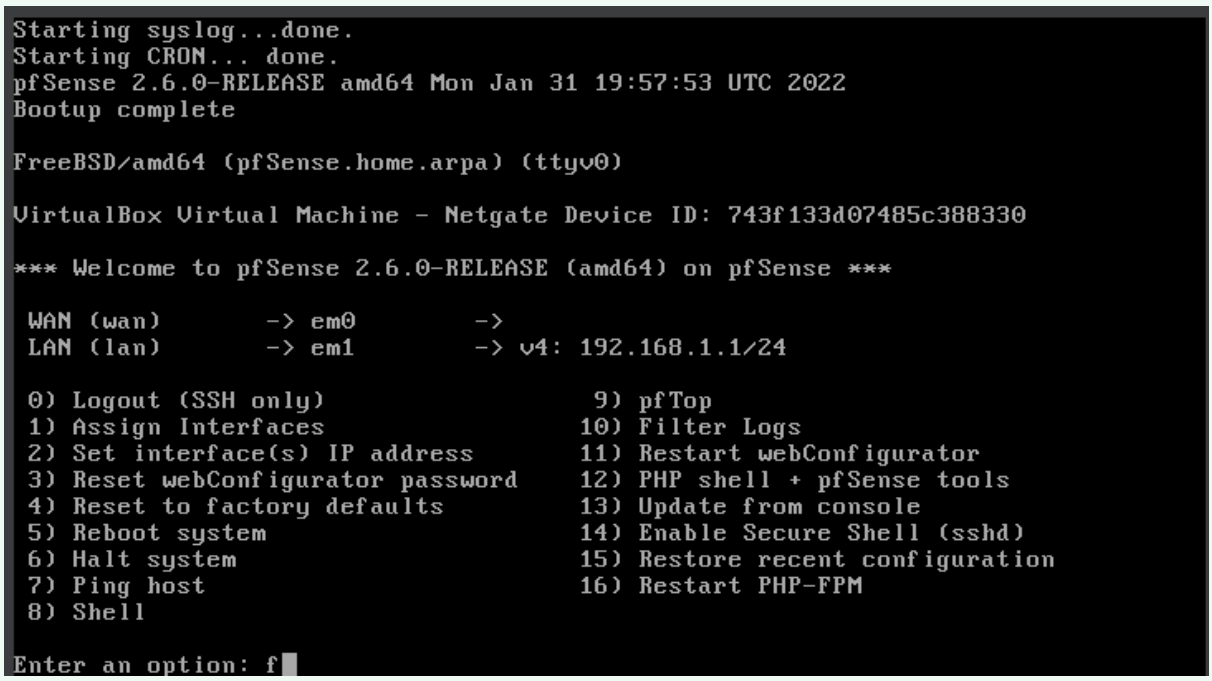
PHASE 1 COMPLETE

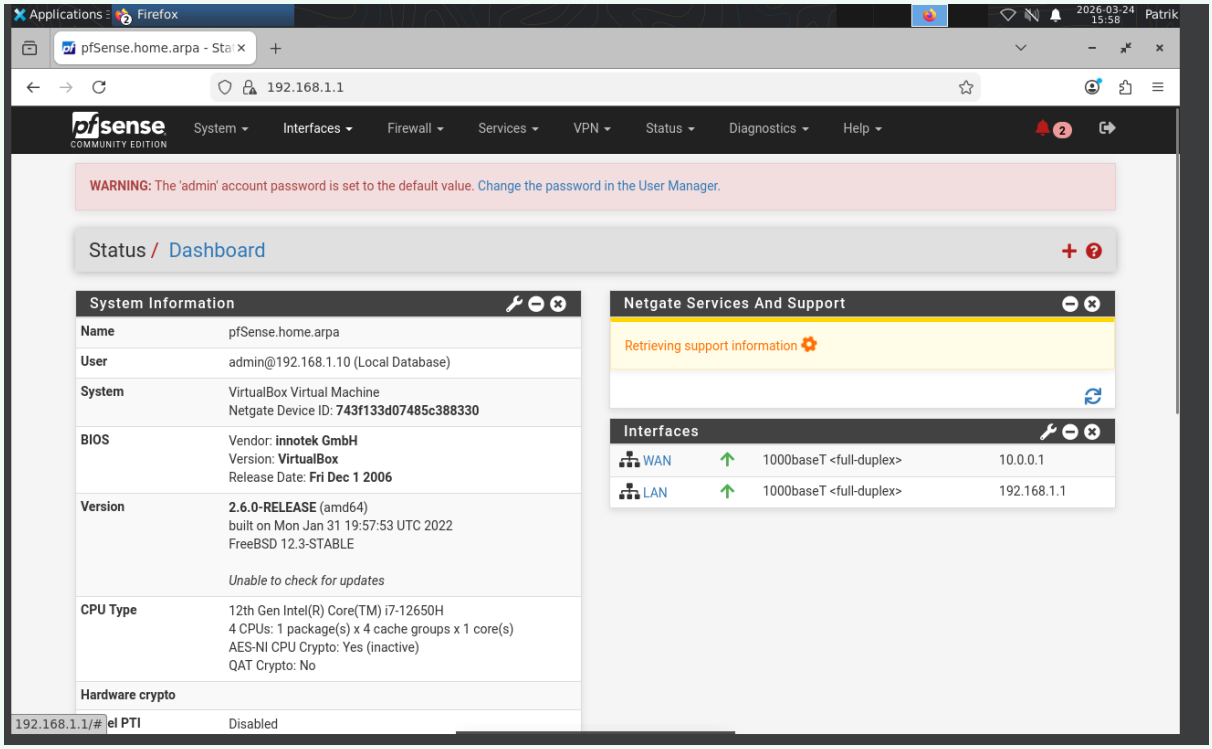
The first phase established the foundational network infrastructure: creating and configuring all virtual machines, installing operating systems, assigning static IP addresses, and verifying baseline connectivity between components.

3.1 pfSense Firewall Installation

pfSense was installed from the official ISO image onto a FreeBSD-type VM with two network adapters. During the first boot, the interface assignment wizard was used to map em0 to the WAN interface (connected to intnet-wan) and em1 to the LAN interface (connected to intnet-lan).

The WAN interface was assigned a static IP of 10.0.0.1/24 using pfSense's console menu (Option 2: Set Interface IP Address). The LAN interface defaulted to 192.168.1.1/24, which was kept as-is since it aligned with the planned addressing scheme.





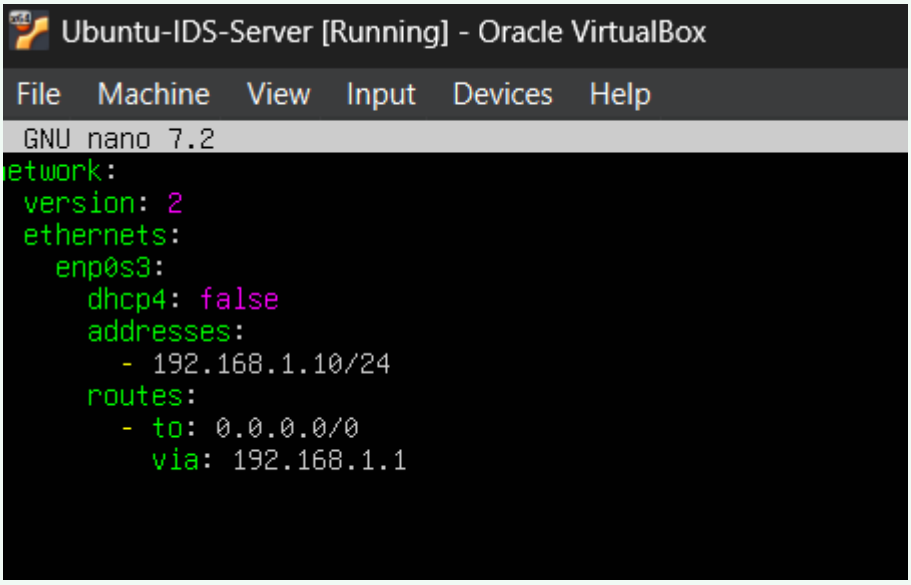
3.2 Ubuntu Server Installation and Configuration

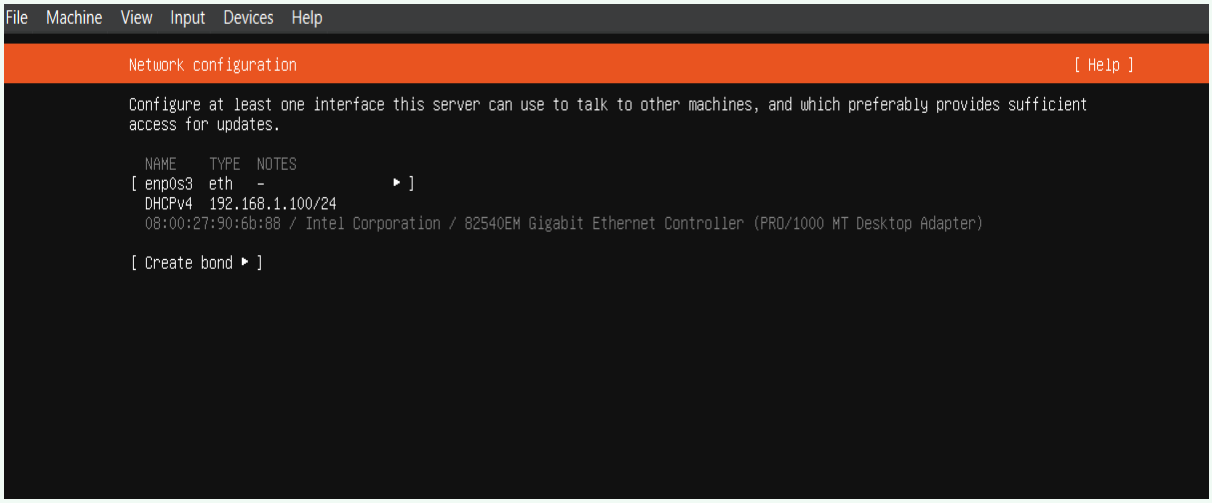
Ubuntu Server 24.04 LTS was installed with the OpenSSH server package enabled during setup. The installation was performed manually (skipping unattended installation) to maintain full control over the configuration process.

A static IP address was configured using Ubuntu's netplan utility:

```
# /etc/netplan/50-cloud-init.yaml
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: false
      addresses:
        - 192.168.1.10/24
      routes:
        - to: 0.0.0.0/0
          via: 192.168.1.1
```

The gateway (192.168.1.1) points to pfSense's LAN interface, establishing it as Ubuntu's route to any network outside the LAN segment.





3.3 Kali Linux Installation and Configuration

Kali Linux was installed with the graphical installer. The network interface was configured with a static IP using the traditional Debian-style interfaces file:

```
# /etc/network/interfaces
auto eth0
iface eth0 inet static
    address 10.0.0.2
    netmask 255.255.255.0
    gateway 10.0.0.1
```

The gateway (10.0.0.1) points to pfSense's WAN interface. From Kali's perspective, pfSense is the router to the internal network — just like a real ISP gateway is the router to the internet.

```
File  Actions  Edit  View  Help
  GNU nano 8.4                                    /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback
auto eth0
iface eth0 inet static
        address 10.0.0.2
        netmask 255.255.255.0
        gateway 10.0.0.1
```

```
┌──(patrik㉿vbox)-[~]
└─$ sudo systemctl restart networking

┌──(patrik㉿vbox)-[~]
└─$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:05:77:7b brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.2/24 brd 10.0.0.255 scope global eth0
       valid_lft forever preferred_lft forever

┌──(patrik㉿vbox)-[~]
└─$
```

3.4 Connectivity Verification

After all three machines were configured, connectivity testing was performed to verify the network topology:

- **Ubuntu → pfSense LAN (192.168.1.1): SUCCESS** — Confirms the LAN segment is operational and Ubuntu can reach its gateway.
- **Kali → pfSense WAN (10.0.0.1): BLOCKED** — Expected behavior. pfSense blocks all WAN inbound traffic by default.

- **Kali → Ubuntu (192.168.1.10): BLOCKED** — Expected behavior. Cross-zone traffic requires explicit firewall rules.

This confirmed that the default deny policy was active and the network segmentation was functioning correctly. All cross-zone communication would need to be explicitly permitted in Phase 2.

4. Phase 2 — Firewall Configuration

PHASE 2 COMPLETE

Phase 2 transformed pfSense from a simple barrier into a precisely configured security gateway. The pfSense web GUI was accessed from Ubuntu Server using Firefox (the XFCE desktop environment was installed specifically for this purpose) at https://192.168.1.1.

4.1 Understanding Default Deny

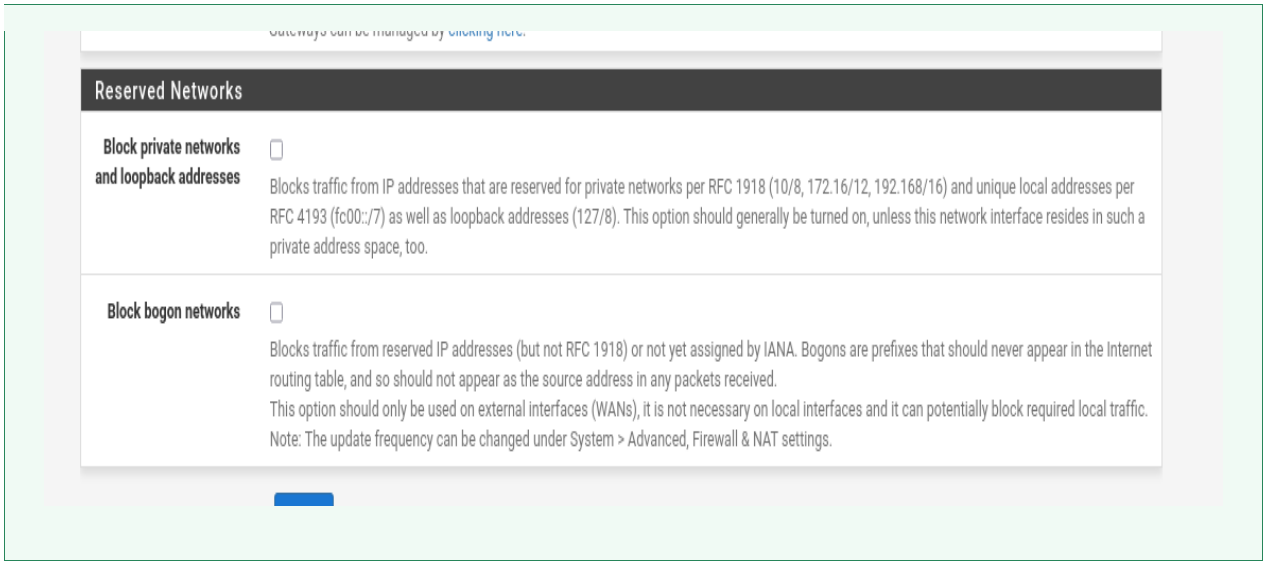
pfSense operates on a fundamental security principle: default deny. Every packet arriving at an interface is evaluated against the ruleset from top to bottom. If no rule matches, the packet is silently dropped. This means the firewall starts in its most secure state — blocking everything — and the administrator must explicitly open only the specific traffic flows that are required.

This is the opposite of a "default allow" approach (which permits everything unless blocked) and is considered an industry best practice for perimeter security.

4.2 Disabling Private Network Blocking

An important discovery during configuration: pfSense includes hidden default rules on the WAN interface that automatically block all traffic from RFC 1918 private IP addresses and bogon ranges. Since this lab uses 10.0.0.0/24 (a private range) on the WAN side to simulate the internet, these rules were silently dropping all of Kali's traffic before any custom rules could be evaluated.

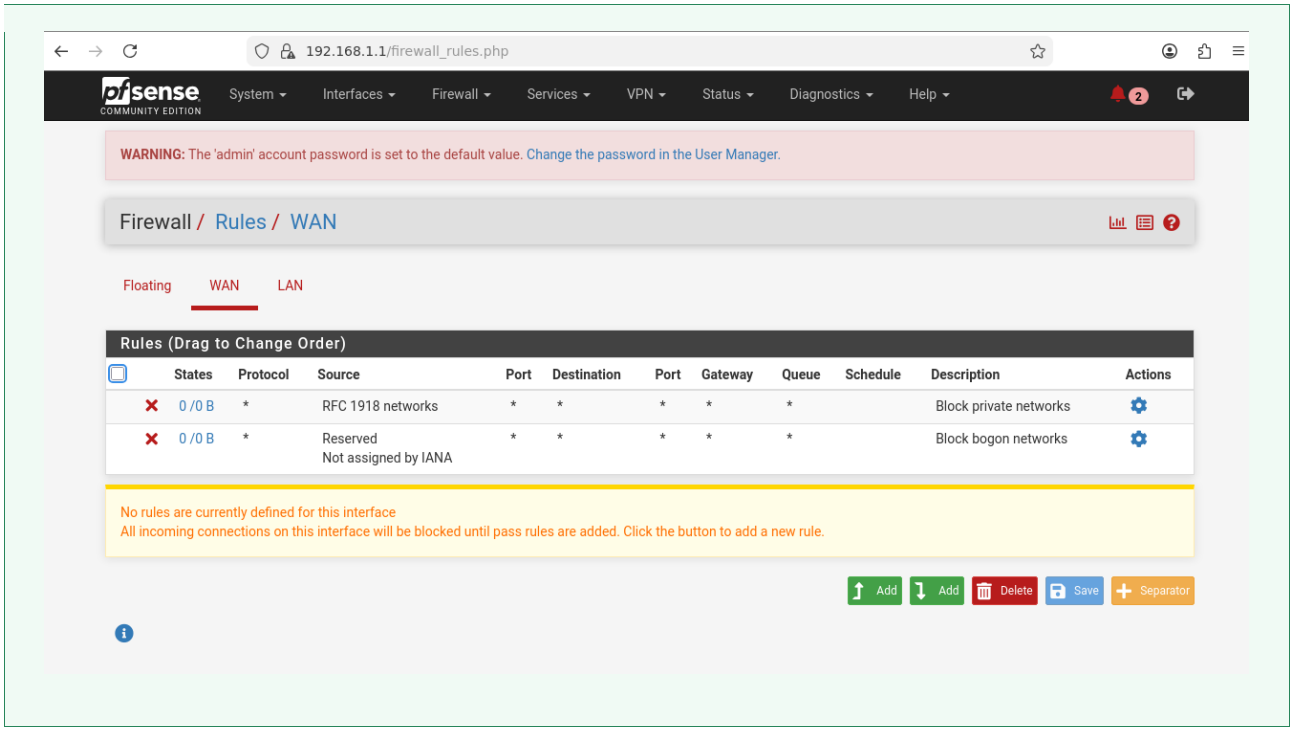
The fix was to disable both "Block private networks" and "Block bogon networks" under Interfaces → WAN. In a production environment with real public IP addresses, these rules would remain active as an additional security layer against spoofed source addresses.

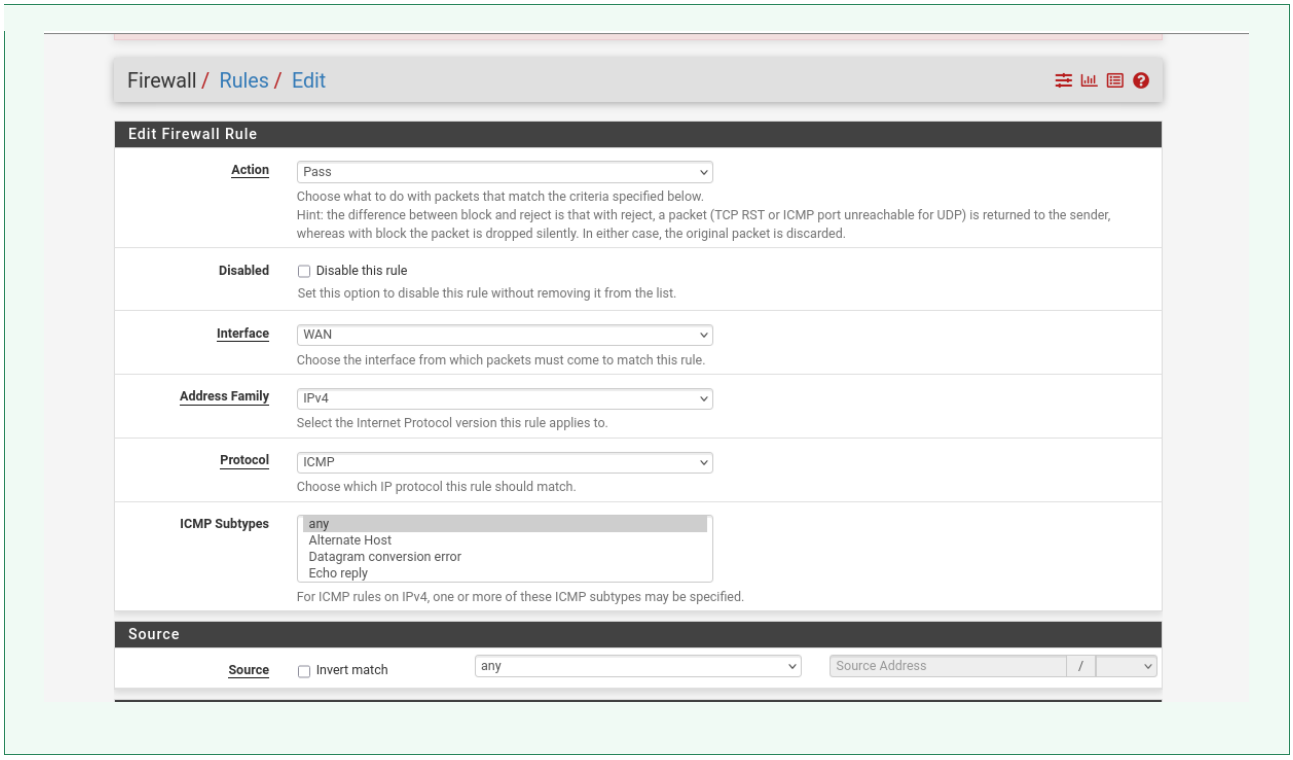


4.3 Firewall Rules — WAN Interface

The following rules were created on the WAN interface, each serving a specific purpose:

#	Action	Protocol	Source	Destination	Port	Description
1	Pass	ICMP	Any	WAN address	—	Allow ping to firewall from outside
2	Pass	TCP	Any	192.168.1.10	22 (SSH)	Allow remote SSH to Ubuntu Server
3	Pass	ICMP	Any	192.168.1.10	—	Allow ping to Ubuntu through firewall
4	Pass	TCP	Any	192.168.1.10	80 (HTTP)	Allow web traffic to Ubuntu Server
—	Block	*	*	*	*	Default deny — everything else is dropped





4.4 NAT Port Forwarding Rules

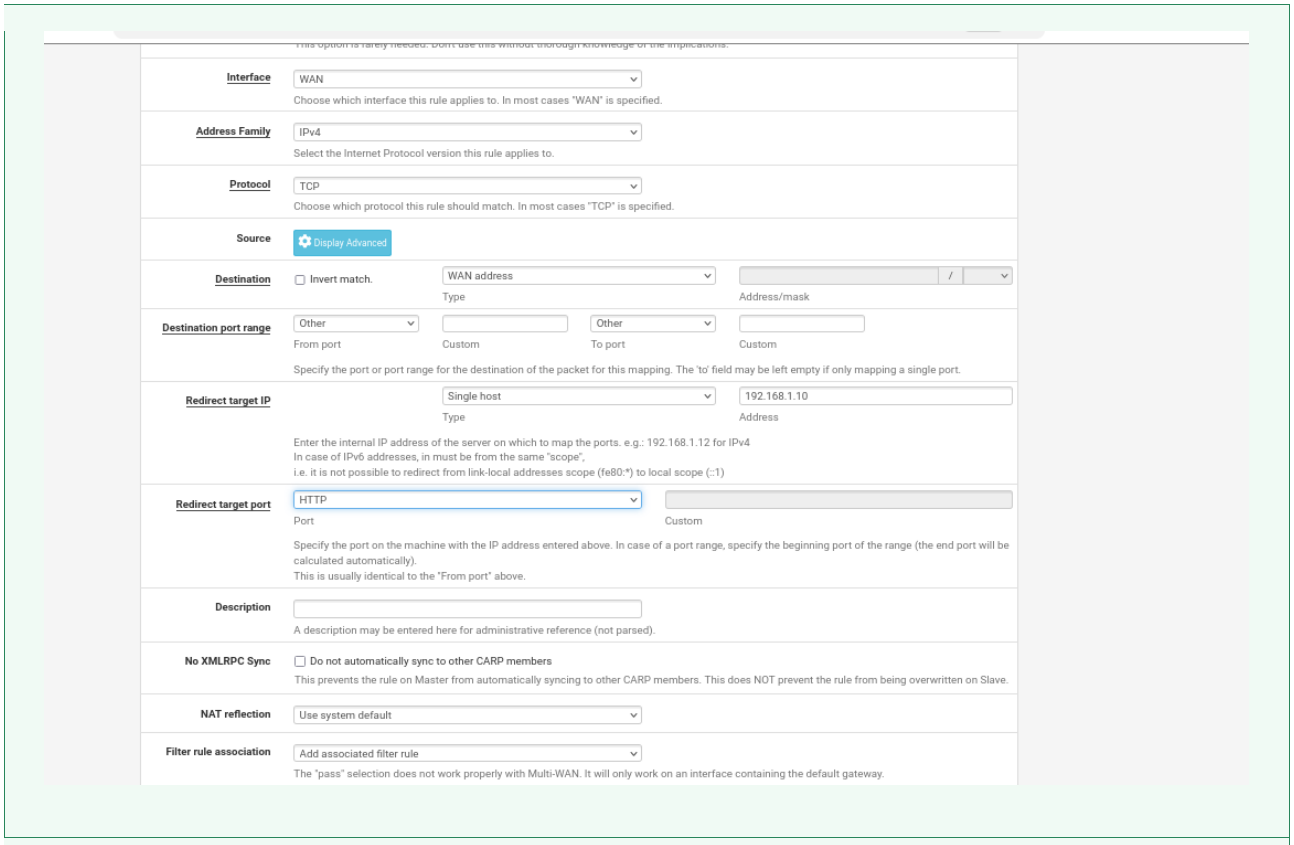
NAT port forwarding was essential because traffic from Kali arrives at pfSense's WAN address (10.0.0.1), but the actual services run on Ubuntu Server (192.168.1.10) behind the firewall. Port forwarding tells pfSense: "When you receive traffic on this port at your WAN address, redirect it to this internal server."

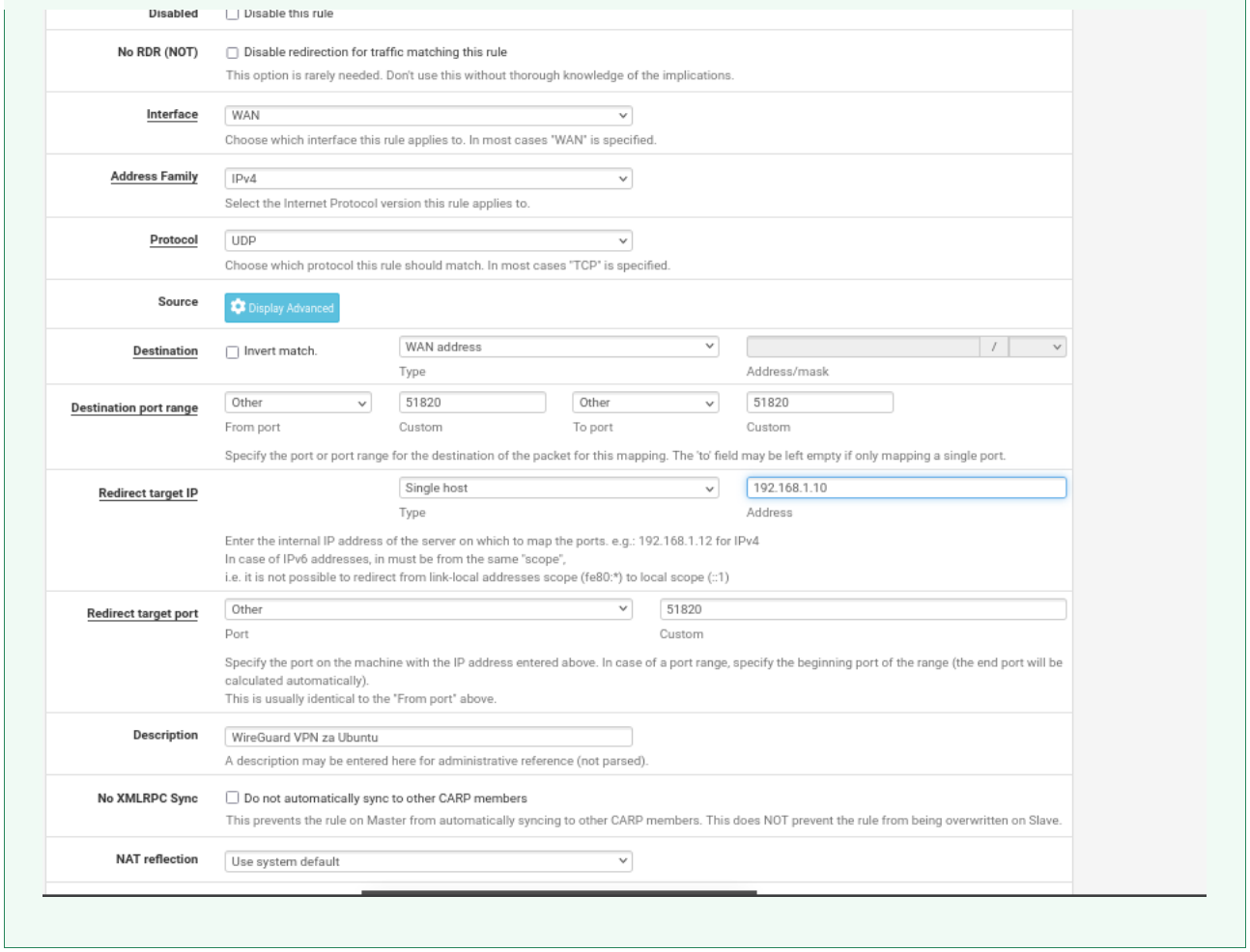
Protocol	WAN Port	Redirect Target	Target Port	Description
TCP	22	192.168.1.10	22	SSH to Ubuntu Server
TCP	80	192.168.1.10	80	HTTP to Ubuntu Server
UDP	51820	192.168.1.10	51820	WireGuard VPN

Edit Redirect Entry

Disabled	☐ Disable this rule
No RDR (NOT)	☐ Disable redirection for traffic matching this rule This option is rarely needed. Don't use this without thorough knowledge of the implications.
Interface	WAN Choose which interface this rule applies to. In most cases "WAN" is specified.
Address Family	IPv4 Select the Internet Protocol version this rule applies to.
Protocol	TCP Choose which protocol this rule should match. In most cases "TCP" is specified.
Source	Display Advanced
Destination	☐ Invert match. WAN address (Type) / Address/mask
Destination port range	SSH (From port) Custom — SSH (To port) Custom

Destination port range	SSH (From port) Custom — SSH (To port) Custom Specify the port or port range for the destination of the packet for this mapping. The 'to' field may be left empty if only mapping a single port.
Redirect target IP	Single host (Type) 192.168.1.10 (Address) Enter the internal IP address of the server on which to map the ports. e.g.: 192.168.1.12 for IPv4 In case of IPv6 addresses, in must be from the same "scope", i.e. it is not possible to redirect from link-local addresses scope (fe80:*) to local scope (::1)
Redirect target port	SSH (Port) Custom Specify the port on the machine with the IP address entered above. In case of a port range, specify the beginning port of the range (the end port will be calculated automatically). This is usually identical to the "From port" above.
Description	A description may be entered here for administrative reference (not parsed).
No XMLRPC Sync	☐ Do not automatically sync to other CARP members This prevents the rule on Master from automatically syncing to other CARP members. This does NOT prevent the rule from being overwritten on Slave.
NAT reflection	Use system default
Filter rule association	Rule NAT View the filter rule


Interface
WAN
Choose which interface this rule applies to. In most cases "WAN" is specified.
Address Family
IPv4
Select the Internet Protocol version this rule applies to.
Protocol
TCP
Choose which protocol this rule should match. In most cases "TCP" is specified.
Source
Display Advanced
Destination
Invert match.
WAN address
Type
Address/mask
Destination port range
Other
From port
Custom
Other
To port
Custom
Specify the port or port range for the destination of the packet for this mapping. The 'to' field may be left empty if only mapping a single port.
Redirect target IP
Single host
Type
192.168.1.10
Address
Enter the internal IP address of the server on which to map the ports. e.g.: 192.168.1.12 for IPv4
In case of IPv6 addresses, in must be from the same "scope",
i.e. it is not possible to redirect from link-local addresses scope (fe80:*) to local scope (::1)
Redirect target port
HTTP
Port
Custom
Specify the port on the machine with the IP address entered above. In case of a port range, specify the beginning port of the range (the end port will be calculated automatically).
This is usually identical to the "From port" above.
Description
A description may be entered here for administrative reference (not parsed).
No XMLRPC Sync
Do not automatically sync to other CARP members
This prevents the rule on Master from automatically syncing to other CARP members. This does NOT prevent the rule from being overwritten on Slave.
NAT reflection
Use system default
Filter rule association
Add associated filter rule
The "pass" selection does not work properly with Multi-WAN. It will only work on an interface containing the default gateway.


Disabled
Disable this rule
No RDR (NOT)
Disable redirection for traffic matching this rule
This option is rarely needed. Don't use this without thorough knowledge of the implications.
Interface
WAN
Choose which interface this rule applies to. In most cases "WAN" is specified.
Address Family
IPv4
Select the Internet Protocol version this rule applies to.
Protocol
UDP
Choose which protocol this rule should match. In most cases "TCP" is specified.
Source
Display Advanced
Destination
Invert match.
WAN address
Type
Address/mask
Destination port range
Other
From port
51820
Custom
Other
To port
51820
Custom
Specify the port or port range for the destination of the packet for this mapping. The 'to' field may be left empty if only mapping a single port.
Redirect target IP
Single host
Type
192.168.1.10
Address
Enter the internal IP address of the server on which to map the ports. e.g.: 192.168.1.12 for IPv4
In case of IPv6 addresses, in must be from the same "scope",
i.e. it is not possible to redirect from link-local addresses scope (fe80:*) to local scope (::1)
Redirect target port
Other
Port
51820
Custom
Specify the port on the machine with the IP address entered above. In case of a port range, specify the beginning port of the range (the end port will be calculated automatically).
This is usually identical to the "From port" above.
Description
WireGuard VPN za Ubuntu
A description may be entered here for administrative reference (not parsed).
No XMLRPC Sync
Do not automatically sync to other CARP members
This prevents the rule on Master from automatically syncing to other CARP members. This does NOT prevent the rule from being overwritten on Slave.
NAT reflection
Use system default

4.5 Verification — Default Deny in Action

To prove the firewall was working correctly, both an allowed and a blocked service were tested from Kali:

HTTP (Port 80) — ALLOWED

A curl request to the web server returned a successful HTML response, confirming that the HTTP firewall rule and NAT port forward were working correctly:

```
$ curl http://10.0.0.1
<!DOCTYPE HTML>
<html><head>...</head>
<body><h1>Directory listing for /</h1>...</body></html>
```

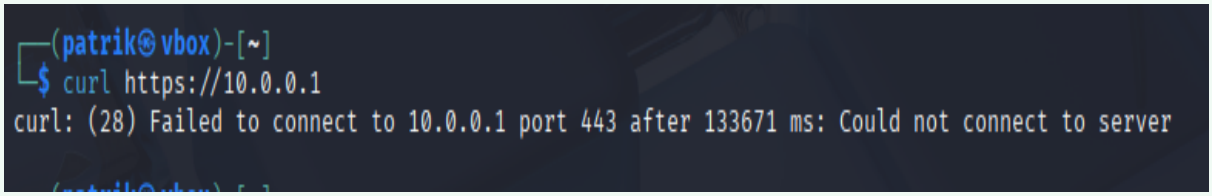
```
┌──(patrik㉿vbox)-[~]
└─$ curl http://10.0.0.1
<!DOCTYPE HTML>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Directory listing for /</title>
</head>
<body>
<h1>Directory listing for /</h1>
<hr>
<ul>
<li><a href=".bash_history">.bash_history</a></li>
<li><a href=".bash_logout">.bash_logout</a></li>
<li><a href=".bashrc">.bashrc</a></li>
<li><a href=".cache/">.cache/</a></li>
<li><a href=".config/">.config/</a></li>
<li><a href=".dmrc">.dmrc</a></li>
<li><a href=".gnupg/">.gnupg/</a></li>
<li><a href=".ICEauthority">.ICEauthority</a></li>
<li><a href=".local/">.local/</a></li>
<li><a href=".profile">.profile</a></li>
<li><a href=".ssh/">.ssh/</a></li>
<li><a href=".sudo_as_admin_successful">.sudo_as_admin_successful</a></li>
<li><a href="Desktop/">Desktop/</a></li>
<li><a href="Documents/">Documents/</a></li>
<li><a href="Downloads/">Downloads/</a></li>
<li><a href="Music/">Music/</a></li>
<li><a href="Pictures/">Pictures/</a></li>
<li><a href="Public/">Public/</a></li>
<li><a href="snap/">snap/</a></li>
<li><a href="Templates/">Templates/</a></li>
<li><a href="Videos/">Videos/</a></li>
</ul>
<hr>
</body>
</html>
```

```
[1]+  Exit 1                  python3 -m http.server 80
patrik@ids-server:~$ sudo python3 -m http.server 80 &
[1] 4186
patrik@ids-server:~$ Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
10.0.0.2 - - [27/Mar/2026 16:00:31] "GET / HTTP/1.1" 200 -
```

HTTPS (Port 443) — BLOCKED

An HTTPS request timed out because no rule or NAT forward exists for port 443. This demonstrates that the default deny policy is actively blocking all traffic not covered by an explicit allow rule:

```
$ curl https://10.0.0.1
curl: (28) Connection timed out after 30001 milliseconds
```



5. Phase 3 — Intrusion Detection System (Suricata)

PHASE 3 COMPLETE

While the firewall controls which traffic is permitted into the network, it cannot inspect the content or behavior of allowed connections. An SSH connection that is permitted by the firewall could be a legitimate administrator — or an attacker brute-forcing passwords. This is where Suricata comes in.

Suricata is an open-source, high-performance Network Intrusion Detection System (IDS) that performs deep packet inspection on network traffic in real time. It compares traffic against a database of known attack signatures and behavioral rules, generating alerts when suspicious patterns are detected.

5.1 Installation

Suricata was installed on Ubuntu Server from the official Ubuntu repositories. This required temporarily enabling a NAT adapter for internet access, installing the package, and then removing the internet connection to maintain lab isolation.

```
patrik@ids-server:~$ sudo systemctl enable suricata
[sudo] password for patrik:
Synchronizing state of suricata.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable suricata
patrik@ids-server:~$ sudo systemctl start suricata
patrik@ids-server:~$
```

5.2 Configuration

Two critical settings were modified in the main configuration file (/etc/suricata/suricata.yaml):

- **HOME_NET** was set to [192.168.1.0/24] — This tells Suricata which network it is protecting. Alerts will be generated when suspicious traffic targets this network.
- **af-packet interface** was set to enp0s3 — The LAN-facing network interface. Suricata captures and analyzes all packets flowing through this interface.

The Suricata rule update tool (suricata-update) was run to download the latest community detection rulesets, providing a baseline of thousands of known attack signatures.

```
vars:
  # more specific is better for alert accuracy and performance
  address-groups:
    HOME_NET: "[192.168.1.0/24,10.0.0.0/8,172.16.0.0/12]"
    #HOME_NET: "[192.168.1.0/24]"
    #HOME_NET: "[10.0.0.0/8]"
    #HOME_NET: "[172.16.0.0/12]"
    #HOME_NET: "any"
```

```
# Linux high speed capture support
af-packet:
  - interface: enp0s3
    # Number of receive threads. "auto" uses the number of cores
    #threads: auto
    # Default clusterid. AF_PACKET will load balance packets based on flow.
    cluster-id: 99
```

```
patrik@ids-server:~$ sudo suricata-update
28/3/2026 -- 15:27:47 - <Info> -- Using data-directory /var/lib/suricata.
28/3/2026 -- 15:27:47 - <Info> -- Using Suricata configuration /etc/suricata/suricata.yaml
28/3/2026 -- 15:27:47 - <Info> -- Using /etc/suricata/rules for Suricata provided rules.
28/3/2026 -- 15:27:47 - <Info> -- Found Suricata version 7.0.3 at /usr/bin/suricata.
28/3/2026 -- 15:27:47 - <Info> -- Loading /etc/suricata/suricata.yaml
28/3/2026 -- 15:27:47 - <Info> -- Disabling rules for protocol pgsql
28/3/2026 -- 15:27:47 - <Info> -- Disabling rules for protocol modbus
28/3/2026 -- 15:27:47 - <Info> -- Disabling rules for protocol dnp3
28/3/2026 -- 15:27:47 - <Info> -- Disabling rules for protocol enip
28/3/2026 -- 15:27:47 - <Info> -- No sources configured, will use Emerging Threats Open
28/3/2026 -- 15:27:47 - <Info> -- Fetching https://rules.emergingthreats.net/open/suricata-7.0.3/emerging.rules.tar.gz.
 100% - 5404806/5404806
28/3/2026 -- 15:27:50 - <Info> -- Done.
28/3/2026 -- 15:27:50 - <Info> -- Loading distribution rule file /etc/suricata/rules/app-layer-events.rules
28/3/2026 -- 15:27:50 - <Info> -- Loading distribution rule file /etc/suricata/rules/decoder-events.rules
28/3/2026 -- 15:27:50 - <Info> -- Loading distribution rule file /etc/suricata/rules/dhcp-events.rules
28/3/2026 -- 15:27:50 - <Info> -- Loading distribution rule file /etc/suricata/rules/dnp3-events.rules
28/3/2026 -- 15:27:50 - <Info> -- Loading distribution rule file /etc/suricata/rules/dns-events.rules
28/3/2026 -- 15:27:50 - <Info> -- Loading distribution rule file /etc/suricata/rules/files.rules
28/3/2026 -- 15:27:50 - <Info> -- Loading distribution rule file /etc/suricata/rules/http-events.rules
28/3/2026 -- 15:27:50 - <Info> -- Loading distribution rule file /etc/suricata/rules/ipsec-events.rules
28/3/2026 -- 15:27:50 - <Info> -- Loading distribution rule file /etc/suricata/rules/kerberos-events.rules
28/3/2026 -- 15:27:50 - <Info> -- Loading distribution rule file /etc/suricata/rules/modbus-events.rules
28/3/2026 -- 15:27:50 - <Info> -- Loading distribution rule file /etc/suricata/rules/nfs-events.rules
28/3/2026 -- 15:27:50 - <Info> -- Loading distribution rule file /etc/suricata/rules/ntp-events.rules
28/3/2026 -- 15:27:50 - <Info> -- Loading distribution rule file /etc/suricata/rules/smb-events.rules
28/3/2026 -- 15:27:50 - <Info> -- Loading distribution rule file /etc/suricata/rules/smtp-events.rules
28/3/2026 -- 15:27:50 - <Info> -- Loading distribution rule file /etc/suricata/rules/stream-events.rules
28/3/2026 -- 15:27:50 - <Info> -- Loading distribution rule file /etc/suricata/rules/tls-events.rules
28/3/2026 -- 15:27:50 - <Info> -- Ignoring file rules/emerging-deleted.rules
28/3/2026 -- 15:27:52 - <Info> -- Loaded 65141 rules.
28/3/2026 -- 15:27:53 - <Info> -- Disabled 14 rules.
28/3/2026 -- 15:27:53 - <Info> -- Enabled 0 rules.
28/3/2026 -- 15:27:53 - <Info> -- Modified 0 rules.
28/3/2026 -- 15:27:53 - <Info> -- Dropped 0 rules.
28/3/2026 -- 15:27:53 - <Info> -- Enabled 136 rules for flowbit dependencies.
28/3/2026 -- 15:27:53 - <Info> -- Backing up current rules.
28/3/2026 -- 15:27:53 - <Info> -- Writing rules to /var/lib/suricata/rules/suricata.rules: total: 65141; enabled: 49280; added: 64760; removed 0; modified:
28/3/2026 -- 15:27:53 - <Info> -- Writing /var/lib/suricata/rules/classification.config
28/3/2026 -- 15:27:53 - <Info> -- Testing with suricata -T.
```

5.3 Custom Detection Rules

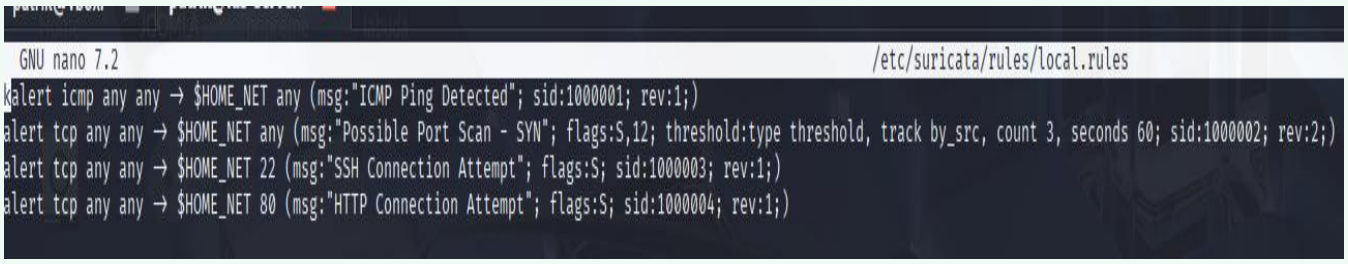
In addition to the community rulesets, four custom rules were written in /etc/suricata/rules/local.rules to detect specific attack patterns relevant to this lab environment:

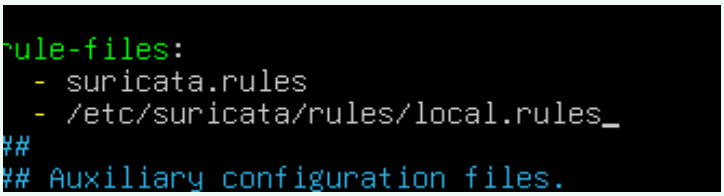
```
# Rule 1: Detect ICMP ping requests to internal network
alert icmp any any -> $HOME_NET any(msg:"ICMP Ping Detected"; sid:1000001; rev:1;)

# Rule 2: Detect potential port scanning (3+ SYN packets in 60 seconds)
alert tcp any any -> $HOME_NET any(msg:"Possible Port Scan - SYN"; flags:S,12;threshold:type threshold, track by_src,count 3, seconds 60;sid:1000002; rev:2;)

# Rule 3: Detect SSH connection attempts to port 22
alert tcp any any -> $HOME_NET 22 (msg:"SSH Connection Attempt"; flags:S; sid:1000003; rev:1;)

# Rule 4: Detect HTTP connection attempts to port 80
alert tcp any any -> $HOME_NET 80 (msg:"HTTP Connection Attempt"; flags:S; sid:1000004; rev:1;)
```





Moving rules inside /etc/suricata/suricata.yaml

5.4 Rule Analysis

Rule 1 (ICMP Detection): Triggers on any ICMP echo request targeting the internal network. While ping is often benign, it is also the first step in network reconnaissance — attackers use it to discover which hosts are alive before launching targeted attacks.

Rule 2 (Port Scan Detection): Uses a threshold mechanism to detect rapid SYN packet bursts from a single source. The threshold (3 SYN packets within 60 seconds to any port) is designed to catch automated scanning tools like Nmap while avoiding false positives from normal TCP connections.

Rule 3 (SSH Monitoring): Alerts on any new TCP connection to port 22. SSH is a high-value target for attackers because it provides full remote command-line access. Monitoring connection attempts helps identify brute-force attacks and unauthorized access attempts.

Rule 4 (HTTP Monitoring): Tracks new TCP connections to port 80. This provides visibility into who is accessing the web server and can help detect web application attacks, directory brute-forcing, or vulnerability scanning.

5.5 Attack Simulation and Detection Results

Three different attack types were launched from Kali to validate Suricata's detection capabilities:

```
# Attack 1: ICMP Ping (Network Reconnaissance)
$ ping -c 3 192.168.1.10

# Attack 2: Nmap SYN Port Scan (Port Discovery)
$ nmap -sS 10.0.0.1

# Attack 3: SSH Connection Attempt (Remote Access)
$ ssh patrik@10.0.0.1
```

Suricata successfully detected and logged all three attack types in /var/log/suricata/fast.log. Each alert entry includes a timestamp, source and destination IP addresses, the protocol, and the matching rule description — providing the security analyst with all the information needed to investigate the incident.

```
patrik@ids-server:~$ sudo cat /var/log/suricata/fast.log
03/29/2026-11:39:34.768561  [**] [1:1000003:1] SSH Connection Attempt [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.0.2:34914 -> 192.168.1.1
0:22
03/29/2026-11:39:35.975411  [**] [1:1000004:1] HTTP Connection Attempt [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.0.2:34914 -> 192.168.1.
10:80
03/29/2026-11:39:37.287473  [**] [1:1000002:2] Possible Port Scan - SYN [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.0.2:34919 -> 192.168.1
.10:80
03/29/2026-11:39:37.287473  [**] [1:1000004:1] HTTP Connection Attempt [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.0.2:34919 -> 192.168.1.
10:80
03/29/2026-11:39:38.542864  [**] [1:1000004:1] HTTP Connection Attempt [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.0.2:34921 -> 192.168.1.
10:80
patrik@ids-server:~$
```

```
patrik@ids-server:~$ sudo systemctl status suricata
● suricata.service - Suricata IDS/IDP daemon
     Loaded: loaded (/usr/lib/systemd/system/suricata.service; enabled; preset: enabled)
     Active: active (running) since Sat 2026-03-28 15:29:09 UTC; 3min 3s ago
       Docs: man:suricata(8)
             man:suricatasc(8)
             https://suricata.io/documentation/
   Main PID: 1070 (Suricata-Main)
      Tasks: 7 (limit: 2265)
     Memory: 551.4M (peak: 551.6M)
        CPU: 31.856s
     CGroup: /system.slice/suricata.service
             └─1070 /usr/bin/suricata -D --af-packet -c /etc/suricata/suricata.yaml --pidfile /run/suricata.pid

Mar 28 15:29:08 ids-server systemd[1]: Starting suricata.service - Suricata IDS/IDP daemon...
Mar 28 15:29:08 ids-server suricata[966]: i: suricata: This is Suricata version 7.0.3 RELEASE running in SYSTEM mode
Mar 28 15:29:09 ids-server systemd[1]: Started suricata.service - Suricata IDS/IDP daemon.
```

6. Phase 4 — WireGuard VPN

PHASE 4 COMPLETE

The final phase adds encrypted remote access to the lab. WireGuard is a modern, high-performance VPN protocol that creates encrypted point-to-point tunnels between endpoints. In this lab, it enables Kali (simulating a remote worker or administrator) to securely access internal LAN resources through an encrypted connection — even though it sits on the untrusted WAN network.

6.1 How WireGuard Encryption Works

WireGuard uses Curve25519 public/private key pairs for authentication and ChaCha20 for symmetric encryption. The process works as follows:

- **Key Generation:** Each endpoint generates its own private/public key pair. The private key never leaves the machine.
- **Key Exchange:** Endpoints share their public keys with each other. This is the only information that needs to be communicated out-of-band.
- **Tunnel Establishment:** When one endpoint wants to communicate, it encrypts the data using the other endpoint's public key. Only the holder of the matching private key can decrypt it.
- **Virtual Network:** A new network (10.10.10.0/24) exists only inside the encrypted tunnel. This is a logical overlay — not a physical network.

6.2 Key Generation

Cryptographic key pairs were generated on both machines using WireGuard's built-in tools:

```
# On Ubuntu Server (VPN Server):
$ wg genkey | sudo tee /etc/wireguard/server_private.key |   wg pubkey | sudo tee /etc/wireguard/server_public.key


# On Kali Linux (VPN Client):
$ wg genkey | sudo tee /etc/wireguard/client_private.key | wg pubkey | sudo tee /etc/wireguard/client_public.key
```

Each machine's public key was then used in the other machine's configuration. Private keys remain strictly on their own machine and are never transmitted over the network.

6.3 Server Configuration (Ubuntu)

```
# /etc/wireguard/wg0.conf (Ubuntu - VPN Server)
[Interface]
PrivateKey = <server_private_key>
Address = 10.10.10.1/24
ListenPort = 51820

[Peer]
PublicKey = <client_public_key>
AllowedIPs = 10.10.10.2/32
```

Interface section: Defines the server's own VPN identity. The server listens on UDP port 51820 and takes the VPN IP address 10.10.10.1.

Peer section: Defines who is allowed to connect. The client's public key authenticates the connection, and AllowedIPs = 10.10.10.2/32 restricts the client to using only its assigned VPN address — preventing IP spoofing within the tunnel.

6.4 Client Configuration (Kali)

```
# /etc/wireguard/wg0.conf (Kali - VPN Client)
[Interface]
PrivateKey = <client_private_key>
Address = 10.10.10.2/24

[Peer]
PublicKey = <server_public_key>
Endpoint = 10.0.0.1:51820
AllowedIPs = 10.10.10.0/24, 192.168.1.0/24
```

Endpoint = 10.0.0.1:51820 — Points to pfSense's WAN address. The NAT port forward rule (configured in Phase 2) redirects UDP 51820 traffic from pfSense to Ubuntu Server at 192.168.1.10.

AllowedIPs = 10.10.10.0/24, 192.168.1.0/24 — This is the routing directive. It tells Kali: "Send traffic destined for these two networks through the VPN tunnel." This means once connected, Kali can access both the VPN network and the entire internal LAN through the encrypted tunnel.

6.5 Tunnel Activation and Verification

The VPN was activated on both machines and tested:

```
# Start VPN Server (Ubuntu):
$ sudo wg-quick up wg0
  [#] ip link add wg0 type wireguard
  [#] ip -4 address add 10.10.10.1/24 dev wg0
  [#] ip link set mtu 1420 up dev wg0

# Start VPN Client (Kali):
$ sudo wg-quick up wg0
  [#] ip link add wg0 type wireguard
  [#] ip -4 address add 10.10.10.2/24 dev wg0
  [#] ip -4 route add 192.168.1.0/24 dev wg0

# Test encrypted tunnel (from Kali):
$ ping -c 3 10.10.10.1
  64 bytes from 10.10.10.1: icmp_seq=1 ttl=64 time=9.58 ms
  64 bytes from 10.10.10.1: icmp_seq=2 ttl=64 time=3.17 ms
  64 bytes from 10.10.10.1: icmp_seq=3 ttl=64 time=2.96 ms
  --- 3 packets transmitted, 3 received, 0% packet loss
```

```
patrik@ids-server:~$ sudo wg-quick up wg0
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.10.10.1/24 dev wg0
[#] ip link set mtu 1420 up dev wg0
patrik@ids-server:~$ _
```

```
└─$ sudo wg-quick up wg0
[sudo] password for patrik:
[#] ip link add dev wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.10.10.2/24 dev wg0
[#] ip link set mtu 1420 up dev wg0
[#] ip -4 route add 192.168.1.0/24 dev wg0
```

```
┌──(patrik㉿vbox)-[~]
└─$ ping -c 3 10.10.10.1
PING 10.10.10.1 (10.10.10.1) 56(84) bytes of data.
64 bytes from 10.10.10.1: icmp_seq=1 ttl=64 time=9.58 ms
64 bytes from 10.10.10.1: icmp_seq=2 ttl=64 time=3.17 ms
64 bytes from 10.10.10.1: icmp_seq=3 ttl=64 time=2.96 ms

--- 10.10.10.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2028ms
rtt min/avg/max/mdev = 2.960/5.238/9.583/3.073 ms
```

6.6 VPN Status Verification

The wg show command confirmed a successful encrypted connection:

```
$ sudo wg show
interface: wg0
  public key: <server_public_key>
  private key: (hidden)
  listening port: 51820

peer: <client_public_key>
  endpoint: 10.0.0.2:54351
  allowed ips: 10.10.10.2/32
  latest handshake: 28 seconds ago
  transfer: 564 B received, 476 B sent
```

The "latest handshake" timestamp confirms that the cryptographic key exchange was successful, and the "transfer" counters show that real encrypted data has been flowing through the tunnel.

```
patrik@ids-server:~$ sudo wg show
interface: wg0
  public key: vvOfQ1bYvP5KuGTEvW+xL4wn2fmomwSLg2jp3Z4liDw=
  private key: (hidden)
  listening port: 51820

peer: xDPe0SF8AbGYJfmgPxaPYd1Shw/nhkak5zp9VlzsKz0=
  endpoint: 10.0.0.2:54351
  allowed ips: 10.10.10.2/32
  latest handshake: 28 seconds ago
  transfer: 564 B received, 476 B sent
```

7. Problems Encountered and Lessons Learned

Building this lab was a deeply iterative process. Many configurations failed on the first attempt, requiring systematic troubleshooting to resolve. Each problem provided practical learning that cannot be gained from textbooks or simulations alone.

7.1 Default Route Conflicts During Package Installation

Problem: When a temporary NAT adapter was added to Ubuntu or Kali for internet access (to install software packages), the system created two competing default routes — one pointing through pfSense (which had no upstream internet connection) and one through the VirtualBox NAT gateway (which did). The operating system would consistently choose the wrong route, sending internet-bound traffic into pfSense where it would reach a dead end.

Symptoms: apt update and apt install would fail with "Unable to connect to archive.ubuntu.com" or "Temporary failure resolving" errors, even though a NAT adapter was enabled and technically had internet access.

Solution: Manually delete the incorrect default route using: sudo ip route del default via <pfSense_IP> dev <interface>. This forced all internet traffic through the NAT adapter.

Lesson: Linux routing tables are evaluated top-down by metric. When a system has multiple network interfaces, route priority management becomes critical. This is a real-world issue that network engineers encounter frequently on multi-homed servers and cloud instances with multiple network interfaces.

7.2 YAML Formatting Errors in Netplan

Problem: Ubuntu's netplan network configuration uses YAML files, which enforce strict formatting rules. During initial setup, the route configuration was written with two separate list items ("- to:" and "- via:") instead of a single route entry with the "via" field nested under "to". This caused netplan to fail with a cryptic "expected sequence" error.

Symptoms: sudo netplan apply would report parsing errors. The network interface would either keep its old configuration or lose its IP address entirely.

Solution: Carefully structure the YAML with proper indentation — "via" must be indented under the same list item as "to", without its own dash prefix. Every space matters, and tabs are not permitted.

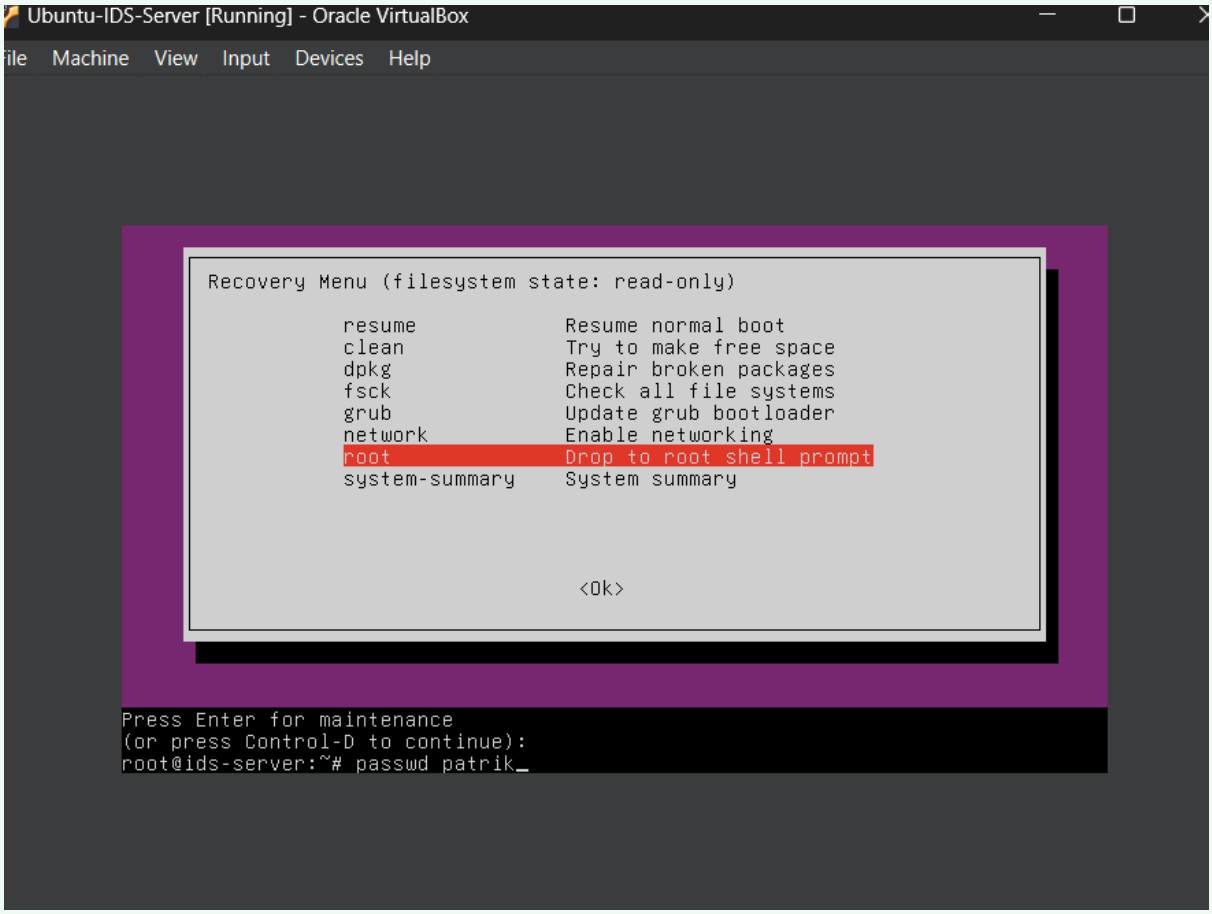
Lesson: Configuration management requires precision. YAML is used extensively in DevOps (Docker Compose, Kubernetes, Ansible) — learning to read and write it correctly is a fundamental skill. Always validate configuration files before applying them in production.

7.3 Forgotten Password and Linux Recovery

Problem: After installing the XFCE desktop environment on Ubuntu Server, the graphical login manager (LightDM) appeared but the installation password could not be recalled. The system was effectively locked out.

Solution: Booted into GRUB recovery mode by holding Shift during startup, selected "Advanced options" then the recovery mode kernel, dropped into a root shell, and reset the password using: passwd <username>.

Lesson: Physical (or virtual) access to a Linux system means full access through recovery mode. This is both a powerful administration tool and a security consideration — in production environments, GRUB should be password-protected and disk encryption should be used to prevent unauthorized access through recovery mode.



8. Tools and Technologies

Tool	Type	Purpose in This Project
Oracle VirtualBox	Type 2 Hypervisor	Virtualization platform hosting all three VMs with isolated internal networks
pfSense	FreeBSD-based Firewall/Router	Network gateway enforcing default deny policy, NAT port forwarding, and inter-zone routing
Ubuntu Server 24.04 LTS	Linux Server OS	Host platform for Suricata IDS and WireGuard VPN server
Kali Linux (Rolling)	Penetration Testing OS	Attack simulation platform with pre-installed security tools (Nmap, SSH, etc.)
Suricata	Network IDS/IPS Engine	Real-time deep packet inspection and alert generation using custom and community rules
WireGuard	VPN Protocol	Modern encrypted tunnel using Curve25519 key exchange and ChaCha20 encryption
Nmap	Network Scanner	Port scanning, service discovery, and network reconnaissance during attack simulation
XFCE + Firefox	Desktop Environment + Browser	Lightweight GUI for accessing pfSense web management interface from Ubuntu Server
Netplan	Network Configuration Tool	Ubuntu's declarative network interface configuration using YAML files

9. Key Takeaways and Skills Demonstrated

This project provided extensive hands-on experience with real security infrastructure. The following competencies were developed and demonstrated:

9.1 Network Engineering

- Designed and implemented a multi-zone network architecture with proper segmentation between trusted and untrusted networks
- Configured static IP addressing across multiple Linux distributions (Ubuntu netplan, Kali interfaces, pfSense console)
- Managed complex routing scenarios including multi-interface systems and temporary NAT connectivity
- Troubleshot Layer 1-3 connectivity issues systematically using ping, arping, ip route, and ip addr

9.2 Security Engineering

- Configured enterprise-grade firewall rules implementing a strict default deny policy on pfSense
- Implemented NAT port forwarding to expose specific internal services while keeping the rest of the network hidden
- Deployed and configured Suricata IDS with both community rulesets and custom-written detection rules
- Wrote targeted IDS rules to detect specific attack patterns including port scans, SSH brute-force attempts, and network reconnaissance
- Implemented WireGuard VPN with proper key management and routing configuration for secure remote access

9.3 Linux System Administration

- Installed and configured Ubuntu Server, Kali Linux, and pfSense (FreeBSD) from scratch
- Managed system services using systemctl (enabling, starting, stopping, and troubleshooting services)
- Performed system recovery using GRUB recovery mode and single-user shell access
- Installed and configured desktop environments (XFCE) and software packages on headless server systems

9.4 Professional Development

- Experienced realistic troubleshooting scenarios that mirror issues encountered in enterprise environments
- Learned to read and interpret error messages, system logs, and network diagnostic output
- Developed the ability to research, diagnose, and resolve issues independently through systematic analysis
- Documented a complex technical project clearly and thoroughly for portfolio and knowledge-sharing purposes

10. Glossary of Terms

Term	Definition
Default Deny	Firewall policy that blocks all traffic unless explicitly allowed by a rule
Defense in Depth	Security strategy using multiple overlapping layers of protection
DHCP	Dynamic Host Configuration Protocol — automatically assigns IP addresses to devices on a network
DNS	Domain Name System — translates human-readable domain names into IP addresses
Firewall	Network security device that monitors and controls incoming/outgoing traffic based on predefined rules
GRUB	Grand Unified Bootloader — boot manager for Linux systems, used for recovery access
ICMP	Internet Control Message Protocol — used by diagnostic tools like ping and traceroute
IDS	Intrusion Detection System — monitors network traffic for suspicious activity and generates alerts
LAN	Local Area Network — a trusted internal network segment
NAT	Network Address Translation — maps private internal IP addresses to public-facing addresses
Netplan	Ubuntu's network configuration abstraction layer using declarative YAML files
Nmap	Network Mapper — open-source tool for network discovery, port scanning, and security auditing
pfSense	Open-source firewall and router platform based on FreeBSD
RFC 1918	Standard defining private IP address ranges (10.x, 172.16-31.x, 192.168.x) not routable on the public internet
SSH	Secure Shell — encrypted protocol for secure remote command-line access to systems
Suricata	Open-source IDS/IPS engine capable of deep packet inspection and real-time threat detection
SYN	Synchronize flag in TCP — the first packet in a three-way handshake to establish a connection
TCP	Transmission Control Protocol — reliable, connection-oriented transport layer protocol
UDP	User Datagram Protocol — lightweight, connectionless transport protocol used by VPNs and DNS
VPN	Virtual Private Network — creates an encrypted tunnel over an untrusted network

WAN	Wide Area Network — an untrusted external network (often the internet)
WireGuard	Modern VPN protocol using Curve25519 key exchange and ChaCha20 symmetric encryption
YAML	Yet Another Markup Language — human-readable data serialization format used in configuration files